



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Motion Estimation

Marco Cagnazzo

Multimedia Communications



Outline

Introduction

Optical flow

Block-matching methods

- Criteria

- Strategy

- Advanced Techniques

Parametric methods

Deep Learning methods

Conclusions



Outline

Introduction

Optical flow

Block-matching methods

Parametric methods

Deep Learning methods

Conclusions



Introduction: movement

- ▶ A video sequence is very rich in information content
- ▶ Movement brings in most of this information
 - ▶ Movement allows objects identification
 - ▶ Image characteristics are coherent along motion trajectories
- ▶ *Motion detection*: binary decision (motion or no motion)
- ▶ *Motion estimation*: measure the movement

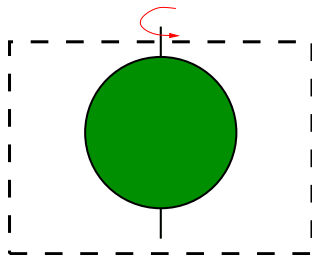


Motion and apparent motion

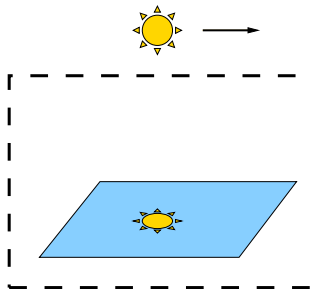
- ▶ 2D motion field:
 - ▶ Projection of physical movement into the image plane
- ▶ Optical flow:
 - ▶ Apparent motion of a luminance pattern
- ▶ The motion field and the optical flow often are the same but this is not necessarily always true



Motion and apparent motion



Purely rotating ball: Null optical flow, real motion is not null



Reflecting surface illuminated by moving source: The optical flow is not null, but there is no physical motion



Motion estimation applications

- ▶ Processing
 - ▶ Segmentation
 - ▶ Denoising
 - ▶ Restoration
 - ▶ Frame-rate up-conversion
 - ▶ Video surveillance / video protection
- ▶ Compression
 - ▶ Current image prediction from encoded images
 - ▶ Trade-off between complexity, precision and coding cost
 - ▶ What is the relationship between the estimated and the “real” motion?
- ▶ Complexity



Outline

Introduction

Optical flow

Block-matching methods

Parametric methods

Deep Learning methods

Conclusions



Variational methods

- ▶ Variational methods are conceived for the estimation of the underlying physical motion
- ▶ They result into a **dense** velocity vector field: one vector per pixel
- ▶ Principles: it is formulated as an optimization problem with *data attachment* and *regularization* terms
- ▶ Typical formulation:

$$\arg \min_{u,v} \{ D[f_0, f_1, (u, v)] + R(u, v) \}$$

- ▶ f_0 and f_1 are the two input images, (u, v) is the estimated motion field with $u(x, y)$ and $v(x, y)$ being the horizontal and vertical velocities
- ▶ D is the data attachment term: it measures how much the estimated field is coherent with the data f_0 and f_1
- ▶ R is the regularization term. It does not depend on data and it is used to penalize solutions that deviated from our *a priori* knowledge of the problem



Optical flow: problem statement

- ▶ Motion is estimated on the basis of brightness variations (Y component)
- ▶ Let us consider a generic point $\mathbf{p}$ in the image coordinates (2D): $\mathbf{p} = [x \ y]^T$
- ▶ A point on the plane corresponds to a point of an object
- ▶ For each point we consider the *trajectory* $\mathbf{x}(t)$ passing through it at a given time as the temporal evolution of the positions of the same point on the physical object
- ▶ We want to estimate the velocity of a generic trajectory at time t and position $\mathbf{p}$: this is the **optical flow**
- ▶ The associate displacement in a given interval (e.g., between two consecutive frames) is referred to as **motion vector field**



Optical flow: the displacement field

- ▶ Let us consider a (point of an) object that, during an interval of duration T (delay), moves from $\mathbf{p} - \mathbf{D}$ to $\mathbf{p}$
- ▶ Its trajectory is $\mathbf{x}(t)$ and

$$\mathbf{x}(t_0) = \mathbf{p} - \mathbf{D}$$

$$\mathbf{x}(t_0 + T) = \mathbf{p}$$

$$\mathbf{D}(\mathbf{p}, t_0, T) = \mathbf{x}(t_0 + T) - \mathbf{x}(t_0) = \begin{bmatrix} c(\mathbf{p}, t_0, T) \\ d(\mathbf{p}, t_0, T) \end{bmatrix} = \begin{bmatrix} c(x, y) \\ d(x, y) \end{bmatrix}$$

- ▶ $\mathbf{D}(\mathbf{p}, t, T)$ is the displacement field and $c(\mathbf{p}, t, T)$, $d(\mathbf{p}, t, T)$ are its components
- ▶ For the sake of simplicity, the dependency of $\mathbf{D}$, c , d on time and delay is not shown explicitly in the following



Optical flow: the constant illumination hypothesis

- ▶ We consider a continuous representation of the video signal, i.e., time and position are continuous variables
- ▶ The *constant illumination hypothesis* (CIH) states that luminance does not change along the motion trajectory $\mathbf{x}(t)$:

$$f(x, y, t+T) = f(x - c(x, y), y - d(x, y), t) \quad (1)$$

- ▶ Practically, we have to consider that Eq. (1) cannot be perfectly fulfilled because of:
 - ▶ spatial and temporal **sampling, aliasing**
 - ▶ **noise** and degradations



Optical flow: the OF equation

The **velocity** field (or, optical flow) is defined as:

$$\mathbf{V}(x, y) = \lim_{T \rightarrow 0} \frac{\mathbf{D}(x, y)}{T} = \begin{bmatrix} u(x, y) \\ v(x, y) \end{bmatrix}$$

From (1), we apply Taylor development, divide by T and compute the limit for $T \rightarrow 0$:

$$\begin{aligned} f(p, t + T) &= f(p, t) - [c(p)f_x(p, t) + d(p)f_y(p, t)] + o[\|\mathbf{D}(p)\|] \\ \frac{f(p, t + T) - f(p, t)}{T} &= -\frac{\mathbf{D} \nabla f}{T} + \frac{o[\|\mathbf{D}(p)\|]}{T} \\ f_t &= -\mathbf{V} \nabla f \end{aligned}$$

$$uf_x + vf_y + f_t = 0$$



Optical flowL the OF equation

► Equation

$$uf_x + vf_y + f_t = 0$$

is called *Optical flow* equation

- It can be discretized for practical applications
- Several motion estimation techniques are based on it
- Nevertheless, we have only one equation and two unknowns, u and v
- Thus, it only allows to compute one component of the velocity field (the one parallel to the image spatial gradient)



Optical flow: solving the OF equation

- ▶ Additional constraints are needed to solve the OF equation
- ▶ Moreover, noise and other degradations make this equation impossible to be perfectly satisfied
- ▶ Solution: we **minimize** the energy of $uf_x + vf_y + f_t$ (instead of setting it to zero) under suitable constraints



Optical flow: the Horn and Schunck algorithm

Horn and Schunck introduced a very reasonable constraint on the **total variation** of the velocity over a region $\mathcal{R}$:

$$\iint_{\mathcal{R}} (uf_x + vf_y + f_t)^2 dx dy = \min$$
$$\iint_{\mathcal{R}} \|\nabla u\|^2 + \|\nabla v\|^2 dx dy \leq \tau$$

Solution: Lagrange multiplier

$$J = \iint_{\mathcal{R}} (uf_x + vf_y + f_t)^2 dx dy +$$
$$\lambda \left[\iint_{\mathcal{R}} (\|\nabla u\|^2 + \|\nabla v\|^2) dx dy - \tau \right]$$



Optical flow: the Horn and Schunck algorithm

Equivalent problem: minimize J on u and v :

$$J = \iint_{\mathcal{R}} (uf_x + vf_y + f_t)^2 + \lambda(\|\nabla u\|^2 + \|\nabla v\|^2) dx dy$$

Now, it is known that in order to minimize:

$$\iint_{\mathcal{R}} \Theta(x, y, \omega, \omega_x, \omega_y) dx dy,$$

we need to impose that:

$$\frac{\partial \Theta}{\partial \omega} - \frac{\partial^2 \Theta}{\partial x \partial \omega_x} - \frac{\partial^2 \Theta}{\partial y \partial \omega_y} = 0$$



Optical flow: the Horn and Schunck algorithm

Setting:

$$\frac{\partial \Theta}{\partial \omega} - \frac{\partial^2 \Theta}{\partial x \partial \omega_x} - \frac{\partial^2 \Theta}{\partial y \partial \omega_y} = 0$$

with $\omega = u$ et $\omega = v$, we get respectively:

$$\lambda \nabla_2 u = (u f_x + v f_y + f_t) f_x$$

$$\lambda \nabla_2 v = (u f_x + v f_y + f_t) f_y$$

Two equations in two unknowns!



Optical flow: the Horn and Schunck algorithm

The Laplacian ∇u can be approximated as $\bar{u} - u$, where $\bar{u}$ is a local average of u values. One gets

$$\begin{cases} \lambda(\bar{u} - u) = (uf_x + vf_y + f_t)f_x \\ \lambda(\bar{v} - v) = (uf_x + vf_y + f_t)f_y \end{cases} \quad \begin{cases} (f_x^2 + \lambda)u + f_y f_t v = \lambda\bar{u} - f_t f_x \\ f_x f_t u + (f_y^2 + \lambda)v = \lambda\bar{v} - f_t f_y \end{cases} \quad \begin{cases} u = \bar{u} - f_x \frac{\bar{u}f_x + \bar{v}f_y + f_t}{\lambda + \|\nabla f\|^2} \\ v = \bar{v} - f_y \frac{\bar{u}f_x + \bar{v}f_y + f_t}{\lambda + \|\nabla f\|^2} \end{cases}$$

However, $\bar{u}$ depends on u and $\bar{v}$ depends on v . In order to find a solution, Horn and Shunk proposed an iterative algorithm, which requires an initialization

$$\begin{cases} u^{n+1} = \bar{u}^n - f_x \frac{\bar{u}^n f_x + \bar{v}^n f_y + f_t}{\lambda + \|\nabla f\|^2} \\ v^{n+1} = \bar{v}^n - f_y \frac{\bar{u}^n f_x + \bar{v}^n f_y + f_t}{\lambda + \|\nabla f\|^2} \end{cases}$$



Optical flow: the Horn and Schunck algorithm

Example





Optical flow: the Horn and Schunck algorithm

Example





Optical flow: the Horn and Schunck algorithm

Example





Optical flow: the Horn and Schunck algorithm

Conclusions

- ▶ Fundamental approach in computer vision
- ▶ Advantages:
 - ▶ Dense optical flow estimation
 - ▶ Smooth and coherent results
 - ▶ Good performance in areas with limited texture
- ▶ Limitations:
 - ▶ Sensitivity to noise
 - ▶ Difficulty with large displacements
 - ▶ Brightness constancy assumption not always valid
- ▶ Applications: object tracking, video compression, motion analysis
- ▶ Foundation for more advanced and modern optical flow estimation algorithms



Outline

Introduction

Optical flow

Block-matching methods

Criteria

Strategy

Advanced Techniques

Parametric methods

Deep Learning methods



Block matching

- ▶ The support is a rectangular block of pixels
- ▶ The motion model is pure translation
 - ▶ As a consequence, the motion is described by a translation vector (motion vector) per block
 - ▶ Affine models (6 parameters per block) have been proposed: they allow to represent zoom and rotation in addition to translation
 - ▶ The estimation of the affine parameters can be less robust than the estimation of pure translation
- ▶ We assume **discrete signals**
- ▶ As a consequences, we will no longer compute a velocity field but a displacement field, that is typically called **motion vector field**



Block matching

- ▶ Block matching is extremely popular in video compression
- ▶ Trade-off between precision and coding cost of motion description
 - ▶ Number of blocks per image (block size)
 - ▶ Number of parameters per block
 - ▶ Number of bits per parameter
- ▶ Complexity can grow very large

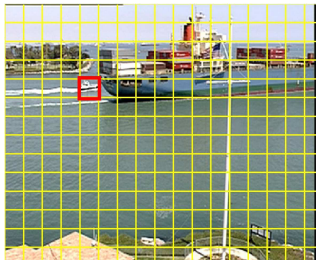


Block matching

Notation

- ▶ Let us consider a $N \times M$ image
- ▶ A block $B_{p,q}$ is defined as the *set of indexes* starting in (p, q) , whose size is $P \times Q$:

$$B_{p,q} = \{p, p+1, \dots, p+P-1\} \times \{q, q+1, \dots, q+Q-1\}$$





Block matching

Notation

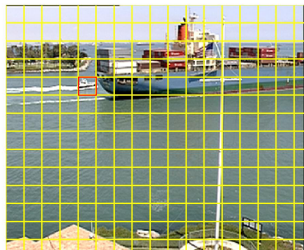
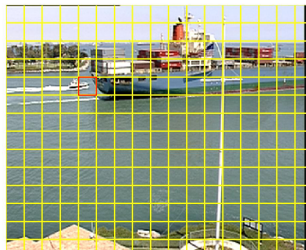
- ▶ We introduce the symbol $\mathbf{f}_k(B_{p,q})$ to indicate the vector of PQ luminance values in the positions $B_{p,q}$ at time k :

$$\mathbf{f}_k(B_{p,q}) = [f(p, q, k), f(p + 1, q, k), \dots, \\ f(p + P - 1, q, k), f(p, q + 1, k), \dots, \\ f(p + P - 1, q + Q - 1, k)]^T$$

- ▶ The block matching consists in computing a *dissimilarity measure* between $\mathbf{f}_k(B_{p,q})$ and $\mathbf{f}_h(B_{p-i,q-j})$
- ▶ The vector (i, j) minimizing this dissimilarity (or distance) is selected

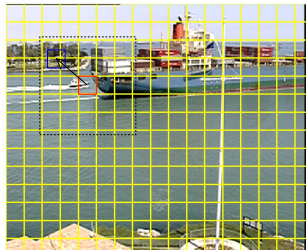


Block matching



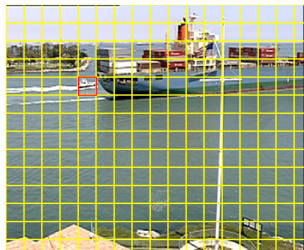
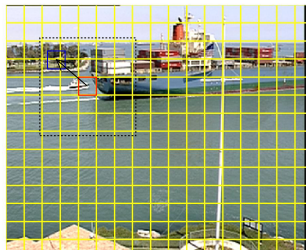


Block matching





Block matching



We compare $\mathbf{f}_k(B_{p,q})$ and $\mathbf{f}_h(B_{p-i,q-j})$



Block matching

Notation

In summary, block matching consists in solving, for each block $B_{p,q}$ of current image k , the following problem:

$$\left(\hat{i}, \hat{j}\right) = \arg \min_{(i,j) \in \mathcal{W}} d\left[\mathbf{f}_k\left(B_{p,q}\right), \mathbf{f}_h\left(B_{p-i, q-j}\right)\right] = \arg \min_{(i,j) \in \mathcal{W}} J(i, j)$$

BM techniques differ in:

- ▶ The minimization criterion d (or J)
- ▶ The set $\mathcal{W}$ of candidate vectors (search window) and the strategy for scanning $\mathcal{W}$
- ▶ The block size and shape



Block matching

- ▶ Therefore, the estimated motion vector field (MVF) (u, v) is a piece-wise constant vector obeying the following:

$$\forall (n, m) \in B_{p,q}, (u(n, m), v(n, m)) = \arg \min_{(i,j) \in \mathcal{W}} d[\mathbf{f}_k(B_{p,q}), \mathbf{f}_h(B_{p-i,q-j})]$$

- ▶ A more precise (but also heavier) notation would be $(u_{h \rightarrow k}(n, m), v_{h \rightarrow k}(n, m))$, highlighting both the *reference* frame k and the *current* frame h
- ▶ For example, when $h = k - 1$ we are estimating the *forward* motion from the previous image
- ▶ We can also estimate the *backward* motion by setting $h = k + 1$
- ▶ However, we will drop the subscript $h \rightarrow k$ when not necessary



Evaluation of the MVF

- ▶ What is a “good” MVF? We need to define some cost and performance criteria for the resulting MVF
- ▶ In order to define the *quality* of the MVF, we compute the **energy of the prediction error**
- ▶ This means that we use the MVF to compute a prediction of image k from image h
- ▶ The prediction is also referred to as **motion-compensated** image or prediction
- ▶ Then we compute the mean square value of the difference (error) between the prediction and the true image h



Evaluation of the MVF

- ▶ In formulas, the motion-compensated (MC-ed) prediction associated to the MVF $(u_{h \rightarrow k}, v_{h \rightarrow k})$ is:

$$\tilde{f}_k(n, m) = f_h(n + u_{h \rightarrow k}(n, m), m + v_{h \rightarrow k}(n, m))$$

- ▶ The prediction error is:

$$e(n, m) = f_k(n, m) - \tilde{f}_k(n, m)$$

- ▶ The MC-ed MSE is:

$$\mathcal{E} = \frac{1}{NM} \sum_{n,m} e^2(n, m)$$

- ▶ As usual, an equivalent metric is the PSNR:

$$\text{PSNR} = 10 \log_{10} \frac{255^2}{\mathcal{E}}$$



Evaluation of the MVF

- ▶ The PSNR is a quality or performance measure of the MVF
- ▶ A cost measure can be the number of bits needed to losslessly encode the MVF
- ▶ It is called the *coding cost*
- ▶ The coding cost depends on the technique used to encode the vectors:
 - ▶ Fixed-length coding
 - ▶ Exp-Golomb coding (possibly, with prediction)
 - ▶ Huffman coding
 - ▶ Arithmetic coding (possibly, context-based)
- ▶ In order to present a measure that is related to the MVF rather than to the technique, we can use the MVF empirical entropy
- ▶ It amounts to estimate the probability of MVF components via their relative frequency in (u, v) , and to use this probability in the entropy formula



Evaluation of the MVF

- ▶ Last but not least, when deciding the parameters of a ME method, we shall consider the computational complexity
- ▶ In order to compute $\arg \min_{(i,j) \in \mathcal{W}} d[\mathbf{f}_k(B_{p,q}), \mathbf{f}_h(B_{p-i,q-j})]$, we need:
 - ▶ For each block $B(p, q)$
 - ▶ and for each candidate vector $(i, j) \in \mathcal{W}$
 - ▶ compute one instance of $d[\mathbf{f}_k(B_{p,q}), \mathbf{f}_h(B_{p-i,q-j})]$
- ▶ Therefore the following choices impact on the computational complexity:
 - ▶ The block size (which determines the number of blocks)
 - ▶ The number of candidate vectors (which depends on the search window *and* on the search strategy)
 - ▶ The cost function (or criterion) d
- ▶ For each of these parameters' choice, there is a trade-off between complexity, quality and coding cost of the MVF
- ▶ Understanding this trade-off is of fundamental importance to design an effective ME technique



Evaluation of the MVF

- ▶ In order to understand this trade-off, let us consider the block-size $P \times Q$
- ▶ A large block size will typically:
 - ▶ reduce the complexity (less blocks)
 - ▶ reduce the coding cost (for the same reasons, less blocks means less vectors to encode)
 - ▶ increase the MSE, because we use a less flexible mathematical model: the MVF is constant on large areas
- ▶ So, what is the ideal block size?



BM criteria

Norm-based criteria

- ▶ Let us see how to choose the matching criterion d
- ▶ We have to choose a **dissimilarity** measure between vectors of luminance values
- ▶ A natural choice is the distance in some metric space
- ▶ That is the norm of the difference:

$$J(i, j) = \|\mathbf{f}_k(B_{p,q}) - \mathbf{f}_h(B_{p-i,q-j})\|_p^p \quad (2)$$

- ▶ Interest cases: $p = 1$ ($\mathcal{L}^1$ norm), $p = 2$ ($\mathcal{L}^2$ norm)



Norm-based criteria: SSD

- ▶ If in Eq. (2) we choose $p = 2$ we obtain the criterion called *Sum of Squared Differences*, SSD:

$$J_{\text{SSD}}(i, j) = \sum_{(n, m) \in B_{p, q}} [f(n, m, k) - f(n - i, m - j, h)]^2$$

- ▶ For a given block size, the SSD minimizes the energy of the MC-ed prediction
- ▶ Therefore, the SSD is the best choice in terms of MVF quality



Norm-based criteria: SSD

However, the SSD has also several disadvantages:

- ▶ It is relatively complex to compute: the criterion d includes a multiplication
- ▶ The square operator highlights errors from outliers, resulting sometimes in irregular vectors, which in turns increase the entropy (coding cost) of the MVF
- ▶ It does not take into account global illumination changes
- ▶ The $p = 1$ norm reduces the impact of the first 2 problems



Norm-based criteria: SAD

- ▶ If in Eq. (2) we choose $p = 1$ we get the *Sum of Absolute Differences* or SAD:

$$J_{\text{SAD}}(i, j) = \sum_{(n, m) \in B_{p, q}} |f(n, m, k) - f(n - i, m - j, h)|$$

- ▶ When using SAD, we look for the prediction error with the smallest $\mathcal{L}^1$ norm
- ▶ When using SSD, we look for the prediction error with the smallest $\mathcal{L}^2$ norm, ie., the smallest energy
- ▶ As a consequence, the prediction error energy in the SAD case can only be larger than the one in the SSD case



Norm-based criteria: SAD

- ▶ However, the SAD motion vector field is generally more regular than the one estimated with SSD
- ▶ The reason lies in **outliers**
- ▶ Outliers are pixels affected by large “noise”
- ▶ I.e., pixels not respecting the constant illumination hypothesis
- ▶ An outlier in position n, m contributes to the cost function with $|f(n, m, k) - f(n - i, m - j, h)|^p$
- ▶ Thus, if $p = 2$ a single outlier (or a few but concentrated in a small area) can affect the estimated motion vector



Norm-based criteria: examples

Reference image



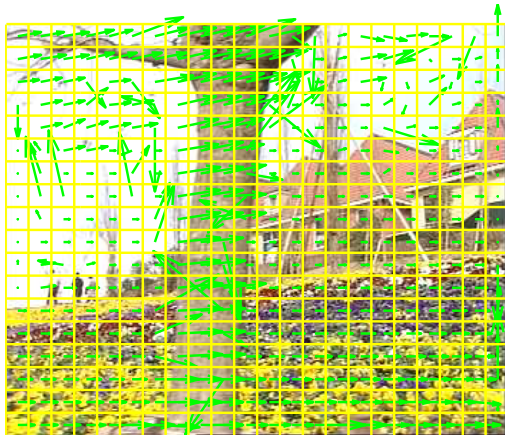
Current image





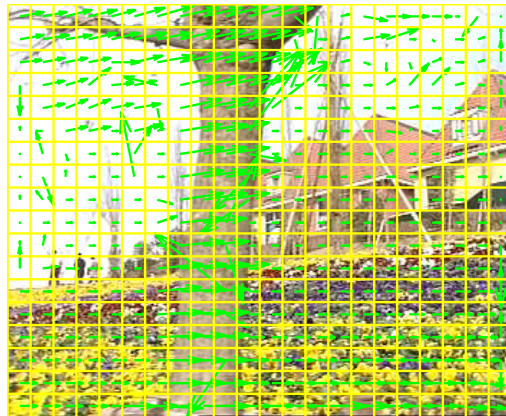
Norm-based criteria: examples

SSD Rate(MV): 2143 bits – PSNR(Pred): 22.46 dB



SSD

SAD Rate(MV): 2103 bits – PSNR(Pred): 22.30 dB



SAD



Norm-based criteria: examples

Motion-compensated image



SSD

Motion-compensated image

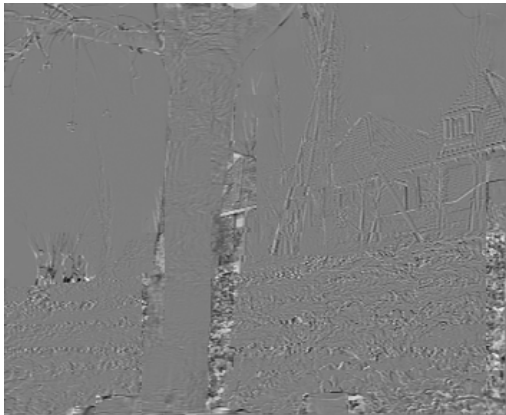


SAD



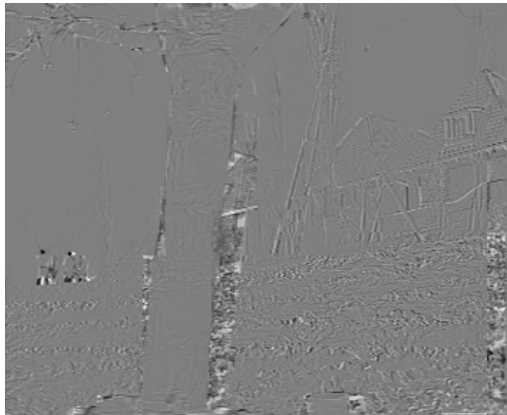
Norm-based criteria: examples

MC error



SSD

MC error

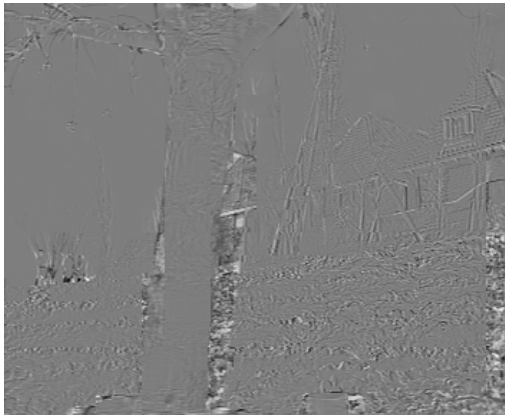


SAD

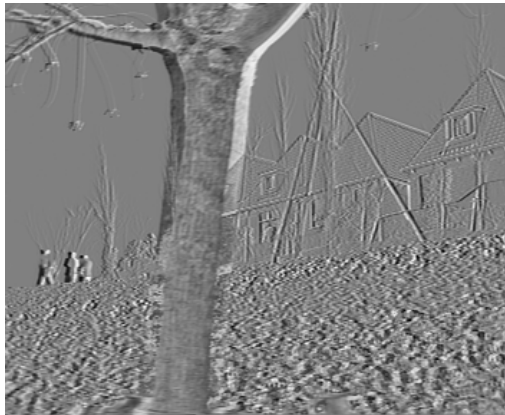


Norm-based criteria: examples

MC error



Difference image





Norm-based criteria

- ▶ These MVF are not always regular: the estimation in homogeneous regions depends finally on noise
- ▶ Advantage: minimization of the error energy
- ▶ Disadvantage: motion artifacts; high coding cost
- ▶ Solution: regularization
 - ▶ Large block sizes, $\mathcal{L}^1$ norm are a form of implicit regularization
 - ▶ We can resort to explicit regularization



Regularized norm-based criteria

- ▶ We modify the criterion (2) by penalizing vectors that are too much different from their neighbors
- ▶ This is accounted for in the function $R(i, j)$
- ▶ Thus we get:

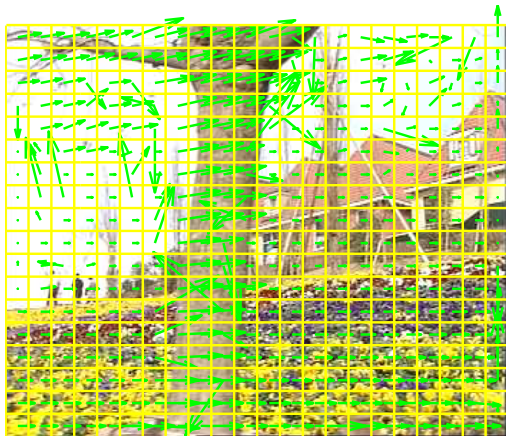
$$J_{\text{REG}}(i, j) = \|\mathbf{f}_k(B_{p,q}) - \mathbf{f}_h(B_{p-i,q-j})\|_p^p + \lambda R(i, j)$$

- ▶ For example, R could be the distance between (i, j) and a vector representing the neighborhood, such as the average or the median



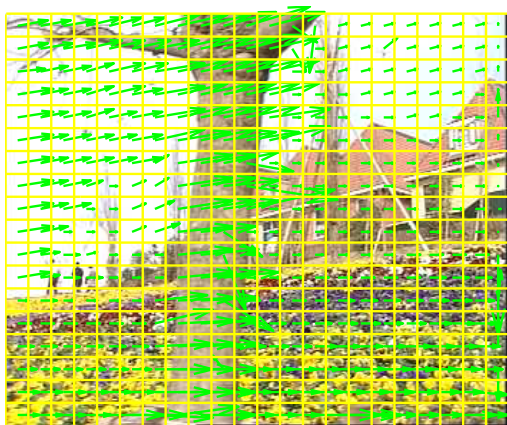
Regularized norm-based criteria: examples

SSD Rate(MV): 2143 bits – PSNR(Pred): 22.46 dB



SSD

SSD Rate(MV): 2008 bits – PSNR(Pred): 22.35 dB



Regularized SSD



Block matching: research strategies

Full search

- ▶ Naive solution: each vector (i, j) must be tested
 - ▶ We compute the criterion J as many times as possible motion vectors:
 $(N - P)(M - Q)$
- ▶ Often, it is enough to consider a smaller region centered in (p, q)
- ▶ We define a *search window* $\mathcal{W}$:

$$\mathcal{W} = \{-A, \dots, -1, 0, +1, \dots, A\} \times \{-B, \dots, -1, 0, +1, \dots, B\}$$

- ▶ Typically, $A = B$. If $n = 2A + 1$, there are n^2 candidate vectors in $\mathcal{W}$
- ▶ The *full search* motion estimation consists in computing J for each vector in $\mathcal{W}$, and taking the one that optimize the criterion



Block matching: research strategies

Fast methods

- ▶ Full search demands n^2 computations in order to find the best vector limited to a displacement of $\pm A$ pixels (horizontally and vertically)
- ▶ Fast methods allows to find motions with the same amplitude $\pm A$ pixels with less than n^2 computations
- ▶ The basic idea is to test a subset of vectors
- ▶ On the other hand, fast methods no longer assure the optimal MV



Block matching: research strategies

Fast methods

General algorithm

1. Initialization : $k = 0$, we choose $\mathcal{W}_1$ and (i_0, j_0)
2. While the stop condition is not satisfied, repeat
 - 2.1 $k = k + 1$
 - 2.2 $(i_k, j_k) = \arg \min_{(i,j) \in \mathcal{W}_k} J(i, j)$
 - 2.3 $\mathcal{W}_{k+1} = \mathcal{F}(\Delta \mathcal{W}_k, (i_k, j_k))$
3. $(\hat{i}, \hat{j}) = (i_k, j_k)$



Block matching: research strategies

Fast methods

- ▶ At each step, a full search is performed only on the subset $\mathcal{W}_k$
- ▶ Then, the search windows is updated according to the results of the block matching at to a given search scheme $\Delta\mathcal{W}_k$ that may vary at each step
- ▶ The stop condition may be based on: the number of iteration; the characteristics of the resulting vector; or on the value of the criterion
- ▶ The number of computations is then $\sum_{k=1}^K \text{card}(\mathcal{W}_k)$ in the case of K iterations



Block matching: research strategies

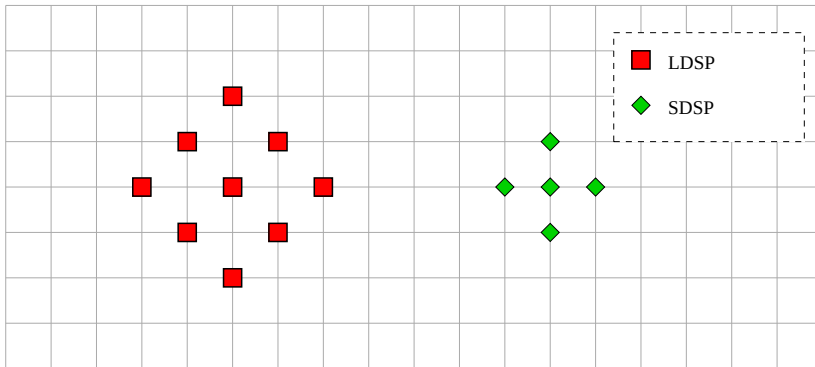
Diamond search

- ▶ The motion pattern size becomes independent from the iteration number
- ▶ We can therefore estimated larger movements than ± 7 pixels
- ▶ A *Large Diamond Search Pattern* and *Small Diamond Search Pattern* are used
- ▶ We start with the LDSP, and we pass to the SDSP only when the central vector of $\mathcal{W}_k$ is selected; otherwise the pattern is moved
- ▶ After one iteration with SDPS the algorithm is stopped



Block matching: research strategies

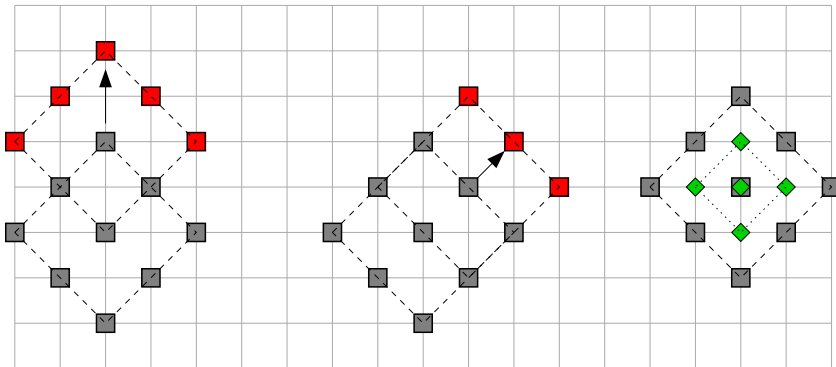
Diamond search





Block matching: research strategies

Diamond search





Block matching: research strategies

Diamond search

- ▶ The LDSP pattern is neither too small (which would have the problem of local minima) not too large such as for 3SS
- ▶ Large improvement with respect to 3SS
- ▶ Quality similar to N3SS with 20% complexity reduction
- ▶ Integrated into MPEG-4 ASP



Block matching: research strategies

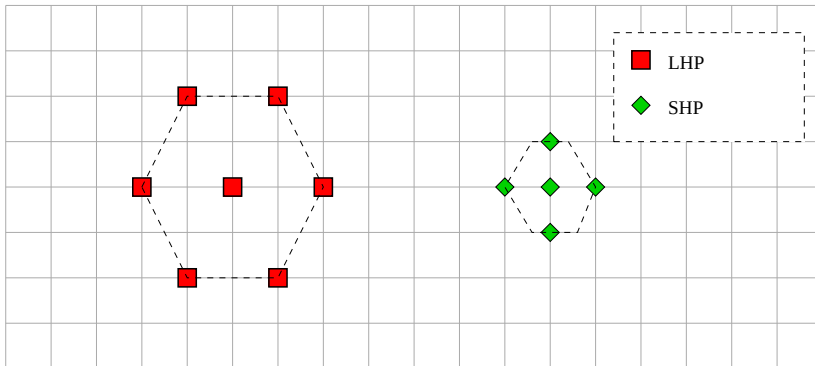
Hexagon search

- ▶ DS: according to the direction we have 5 or 3 computations per iteration
- ▶ Displacement speed: 2 pixels in H/V, only $\sqrt{2}$ in diagonal
- ▶ With a more regularly shaped pattern we could have a uniform speed and less computations



Block matching: research strategies

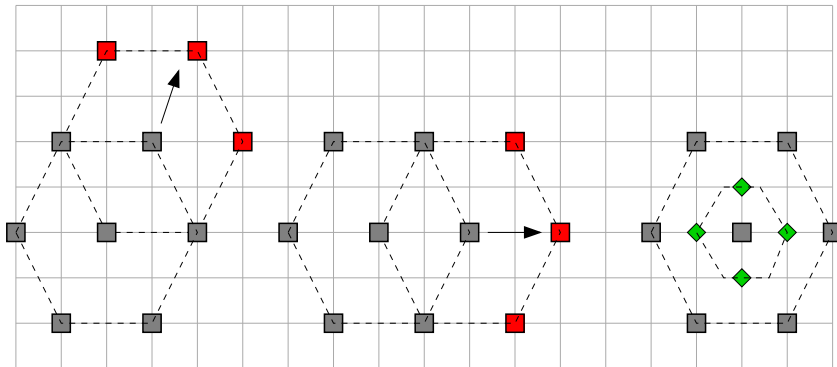
Hexagon search





Block matching: research strategies

Hexagon search





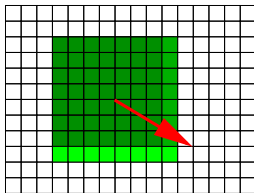
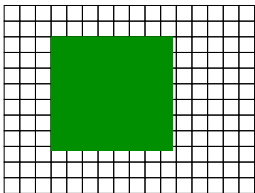
Block matching: research strategies

Hexagon search

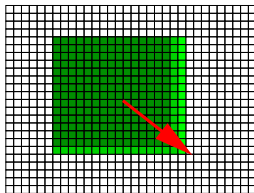
- ▶ Speed: 2 pixel in H, $\sqrt{5} \approx 2.2$ in diagonal
- ▶ Three new computations at each iteration (four in the case of SHP)
- ▶ Complexity reduction of roughly 40% with respect to DS for the same estimation quality
- ▶ Integrated into H.264



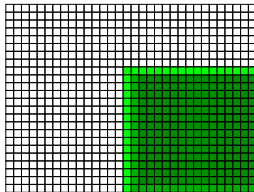
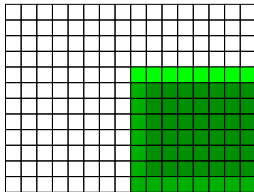
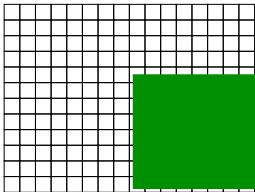
Block matching: subpixel precision



$v=(5,3)$



$v=(5.5,2.5)$

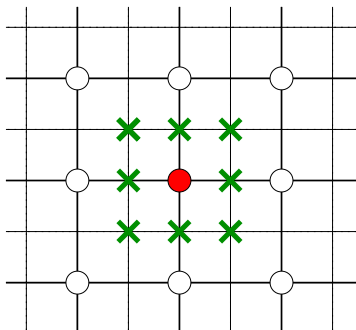




Block matching: subpixel precision

Hierarchical approach:

- ▶ First we find $(\hat{i}, \hat{j}) \in \mathbb{Z}^2$;
- ▶ then we test $(\hat{i} \pm \frac{1}{2}, \hat{j} \pm \frac{1}{2})$;
- ▶ then we verify the 1/4-pixel neighbors, etc.

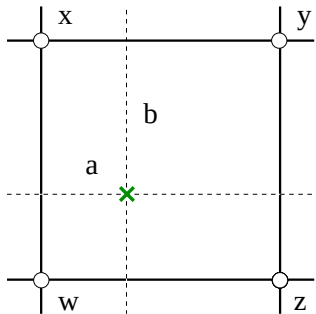


- Full-pixel positions
- Selected full-pixel vector
- × Tested half-pixel positions



Block matching: subpixel precision

In order to evaluate $f(n \pm \alpha, m \pm \beta, k)$ an interpolation is computed



$$f(n, m) = x \quad f(n+1, m) = y$$

$$f(n, m+1) = z \quad f(n+1, m+1) = w$$

Bilinear interpolation: horizontal and vertical average of pixels

$$f(n+a, m+b) = (1-a)(1-b)x + a(1-b)y + (1-a)bz + abw$$

More complex interpolation techniques have been proposed (high order filters)



Block matching: variable block size

- ▶ Underlying hypothesis in BM: motion is homogeneous in a block
- ▶ It is not true if a block contains object's borders
- ▶ Solution 1: smaller blocks
 - ▶ Improved precision
 - ▶ Increased complexity
 - ▶ Increased coding cost (number of motion vectors per image)
- ▶ Solution 2: variable-size blocks
 - ▶ Additional coding cost and complexity: only if necessary
 - ▶ Hierarchical approach: a block is split if the value of the criterion is too big
 - ▶ This solution is adopted in recent video coding standards



Outline

Introduction

Optical flow

Block-matching methods

Parametric methods

Deep Learning methods

Conclusions



Definitions

- ▶ Using parametric models means that the motion vector field is modeled as a close-form function of the pixel position
- ▶ The degrees of freedom are the *parameters* of the mathematical function
- ▶ The parametric model can be unique for all the scene (global motion estimation); or one can use a different parametric model per region
- ▶ In turns, the region can be an object (in this case, we first need to *segment* the image to find the objects) or a pre-determined set of pixels, e.g., a block



Definitions

- ▶ Block-matching methods can be seen as a special form of parametric methods, where the MVF is a block-wise constant function, with two free parameters per block (pure translation)
- ▶ The regions are the blocks
- ▶ More general models can be conceived
 - ▶ Less parameters → Robust estimation, but less generality, can miss the complexity of motion
 - ▶ More parameters → Prone to overfitting, more complex but also more general



Translational model

- ▶ The simplest motion model is the translational one
- ▶ This model correspond to the case of a rigid object that translates orthogonally to the optic axis without rotations
- ▶ Let $\mathbf{v}(p)$ be the instantaneous velocity in position $p = [x, y]^T$ on the image plane
- ▶ Components $\mathbf{v}_x$ and $\mathbf{v}_y$ of $\mathbf{v}$ do not vary with p , they depend on camera parameters and on object motion
- ▶ In other words, the model is a constant vector (two parameters). We can write:

$$\mathbf{v}(p) = \begin{bmatrix} \mathbf{v}_x \\ \mathbf{v}_y \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

- ▶ This model is only characterized by two parameters, b_1 et b_2



Affine model

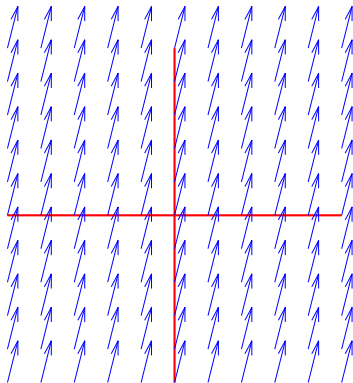
- ▶ The affine model is more complex, but it can represent a large variety of motions
- ▶ Its equation is:

$$\mathbf{v}(p) = \mathbf{b} + Bp = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} + \begin{bmatrix} b_3 & b_4 \\ b_5 & b_6 \end{bmatrix} p$$

- ▶ It includes translation as a special case when $B = 0$
- ▶ It is based on 6 parameters, two for $\mathbf{b}$ and four for B
- ▶ The relatively small number of parameters makes their estimation robust



Affine models: examples



Translation:

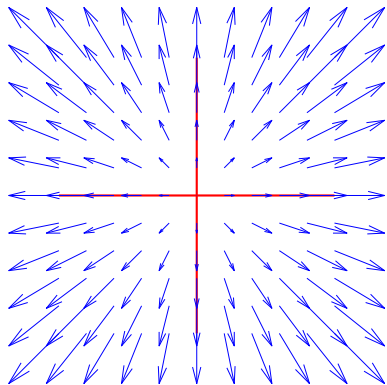
$$\mathbf{b} = \begin{bmatrix} 0.5 \\ 2 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$\mathbf{v}_x = 0.5$$

$$\mathbf{v}_y = 2$$



Affine models: examples



Zoom in:

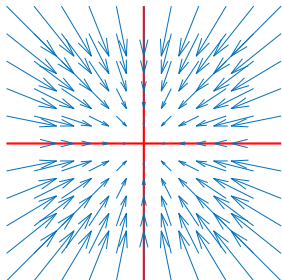
$$\mathbf{b} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$$

$$\mathbf{v}_x = 0.5x$$

$$\mathbf{v}_y = 0.5y$$



Affine models: examples



Zoom out:

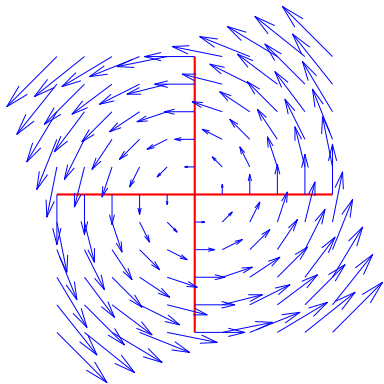
$$\mathbf{b} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} -0.5 & 0 \\ 0 & -0.5 \end{bmatrix}$$

$$\mathbf{v}_x = -0.5x$$

$$\mathbf{v}_y = -0.5y$$



Affine models: examples



Rotation:

$$\mathbf{b} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 0 & -0.5 \\ 0.5 & 0 \end{bmatrix}$$

$$v_x = -0.5y$$

$$v_y = 0.5x$$



Parametric motion estimation

- ▶ Once we decide for a motion model, we need to estimate the value of the parameters on each region
- ▶ Remember, a region is a set of pixel where we use an identical motion model: can be an object or a fixed set, like a block
- ▶ If the region is the full image, the ME is called *Global ME*
- ▶ Typically this is applied for estimating camera motion, which physically involves:
 - ▶ Motion along the 3 axes
 - ▶ Rotation around the 3 axes
- ▶ However, the parametric model is often much simple, based on translational or affine models



Indirect estimation

- ▶ How to estimate the parameters of the model?
- ▶ A first approach consists in first estimating a dense field (e.g. with optical flow), and then deduce the global motion from it solving an optimization problem, typically with least square techniques
- ▶ Let $u(n, m)$ et $v(n, m)$ be the estimated dense field components
- ▶ Let $u_\pi(n, m)$ et $v_\pi(n, m)$ be some dense fields depending on the parameter set π
 - ▶ For example, for affine models, $\pi = [b_1 \ b_2 \ b_3 \ b_4 \ b_5 \ b_6]$
- ▶ Indirect estimation amount to:

$$\pi^* = \arg \min_{\pi} \sum_{n, m \in \mathcal{R}} [u(n, m) - u_\pi(n, m)]^2 + [v(n, m) - v_\pi(n, m)]^2$$



Indirect estimation

- ▶ Indirect estimation looks for the parameters that make the model the closest to the estimated MVF
- ▶ This distance minimization may be solved with a gradient descent
- ▶ Disadvantage:
 - ▶ Strong dependency from the first dense estimation
 - ▶ It needs a support region $\mathcal{R}$ with homogeneous motion



Direct estimation

- ▶ With Direct Estimation, the parameters are introduced in the estimation phase
- ▶ For example, OF equation becomes:

$$\pi^* = \arg \min_{\pi} \sum_{n,m \in \mathcal{R}} [u_{\pi}(n, m)f_x(n, m) + v_{\pi}(n, m)f_y(n, m) + f_t(n, m)]^2$$

- ▶ Another approach consists in minimize the SSD or the SAD computed on the parametrized MVF:

$$\pi^* = \arg \min_{\pi} \sum_{n,m \in \mathcal{R}} [e(n, m)]^2 \quad \text{with}$$

$$e(n, m) = f(n - u_{\pi}(n, m), m - v_{\pi}(n, m), t - 1) - f(n, m, t)$$

- ▶ Block-matching based on SAD can be seen as a special case of direct parametric estimation where the motion model is a pure translation, *i.e.* special case of affine model with $B = 0$



Outline

Introduction

Optical flow

Block-matching methods

Parametric methods

Deep Learning methods

Conclusions



Deep Learning approach

- ▶ Does not require a model or estimation criteria
- ▶ Requires a deep network architecture
- ▶ Usually based on Convolutional Neural Networks
- ▶ Requires a training database



Deep Learning approach

- ▶ CNNs are effective for solving computer vision problems
- ▶ But optical flow estimation requires precise per-pixel localization and feature matching
- ▶ Can CNN learn this?
- ▶ Training problems: only small dataset with ground truth (GT) are available
 - ▶ Data augmentation
 - ▶ Synthetic data with GT



Deep Learning approach

Evolution and Breakthroughs

- ▶ **FlowNet (2015) / FlowNet2 (2017)**
 - ▶ First end-to-end CNN for optical flow
 - ▶ Pioneered direct dense flow field regression
 - ▶ Reference: Dosovitskiy et al., "FlowNet: Learning Optical Flow with Convolutional Networks," ICCV 2015
- ▶ **PWC-Net (2018)**
 - ▶ Introduced pyramid, warping, and cost volume concepts
 - ▶ Set a new standard for efficiency and accuracy
 - ▶ Reference: Sun et al., "PWC-Net: CNNs for Optical Flow Using Pyramid, Warping, and Cost Volume," CVPR 2018



Deep Learning approach

Evolution and Breakthroughs

▶ RAFT (2020)

- ▶ Innovative iterative refinement approach
- ▶ All-pairs correlation volume for improved accuracy
- ▶ Reference: Teed and Deng, "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow," ECCV 2020

▶ GMA (2021)

- ▶ Enhanced RAFT with global motion aggregation
- ▶ Better handling of long-range dependencies
- ▶ Reference: Jiang et al., "Learning to Estimate Hidden Motions with Global Motion Aggregation," ICCV 2021



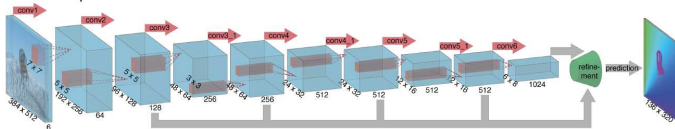
Deep Learning approach

- ▶ CNNs are good at learning complex input/output relationships
- ▶ So the basic idea is to train a CNN end-to-end with a couple of images as input and the corresponding optical flow as output (GT)
 - ▶ Simple architecture (FlowNetS): stack the images, use a rather generic CNN, learn the I/O
 - ▶ Specialized architecture (FlowNetC): learn features with two identical CNNs on the two images, match them at later stage
 - ▶ Complex architecture (FlowNet2): use a complex combination of FlowNetS and FlowNetC to further improve the estimation

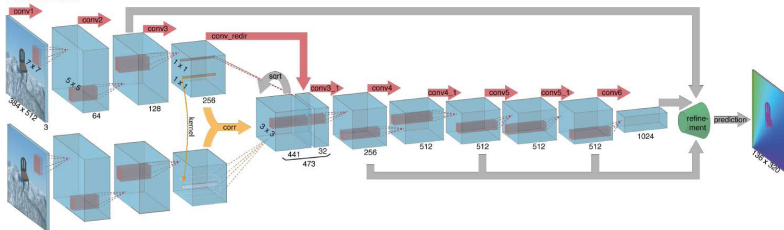


FlowNet

FlowNetSimple



FlowNetCorr



Philipp Fischer, Alexey Dosovitskiy, Eddy Ilg, Philip Hausser, Caner Hazirbas, Vladimir Golkov, Patrick van der Smagt, Daniel Cremers, Thomas Brox,
“FlowNet: Learning Optical Flow with Convolutional Networks” arXiv:1504.06852

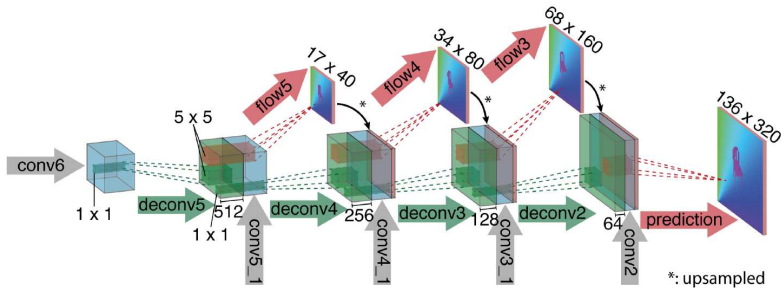


FlowNet, Refinement

- ▶ Pooling in CNNs is necessary to make the training feasible
- ▶ but it shrinks the output size
- ▶ Solution “upconvolutional” layers (aka, deconvolutional layers)
- ▶ They do unpooling (ie. extend the feature maps) and convolution
- ▶ Concatenate with feature maps from the first part of the CNN
- ▶ 4 upconv layers followed by variational refinement based on contour estimation

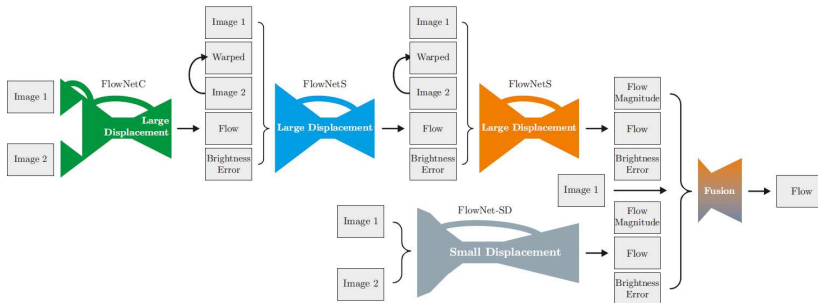


FlowNet, Refinement





FlowNet2.0



Eddy Ilg, Nikolaus Mayer, Tonmoy Saikia, Margret Keuper, Alexey Dosovitskiy, Thomas Brox, "FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks", arXiv:1612.01925



FlowNet2.0

- ▶ FlowNet2.0 achieve better results than FlowNet thanks to a more complex architecture and a much improved training
- ▶ Results of FlowNet2.0 are among the best





Conclusion

- ▶ Rapid progress in neural network-based optical flow
- ▶ Trends: Increasing accuracy, Improved efficiency, Handling of challenging scenarios
- ▶ Future directions: Real-time performance, generalization, integration with other vision tasks



Outline

Introduction

Optical flow

Block-matching methods

Parametric methods

Deep Learning methods

Conclusions



Motion Estimation: Wrap-up

- ▶ Motion Estimation allows to extract motion information from videos
- ▶ It focus on *optical flow* rather than physical motion (other sensors can be used to assess the physical motion)
- ▶ Block-matching: conceptually simple, commonly used for video compression
- ▶ Neural networks: emerging framework, very effective for analysis tasks

